Maven is an open-source build automation and project management tool mainly used for Java-based projects.

It helps developers:

- automate project builds
- manage dependencies
- create JAR/WAR files
- standardize project structure
- simplify deployment

Maven is developed by Apache and written in Java.

Why Maven is Used

Before Maven:

- developers manually downloaded JAR files
- manually managed dependencies
- manually compiled source code
- manually packaged applications

This created:

- dependency conflicts
- inconsistent project structures
- build failures
- deployment difficulties

Maven solves these issues by automating the complete build process.

Maven Workflow

```
None
Developer
 ↓
Writes pom.xml
 ↓
Maven Reads pom.xml
 ↓
Downloads Dependencies
 ↓
Compiles Source Code
```

```
 ↓
Runs Tests
 ↓
Creates Artifact
 ↓
Deployment
```

Maven Architecture

```
None
pom.xml
 ↓
Maven Core
 ↓
Plugins
 ↓
Lifecycle
 ↓
Repositories
 ↓
Artifact Generation
```

Maven Project Structure

```
None
employee-project/
│
├── pom.xml
│
├── src/
│   ├── main/java
│   └── test/java
│
└── target/
```

Pom.xml

Explanation

POM means: Project Object Model

It is the main configuration file in Maven.

It stores:

- dependencies
- plugins
- build configurations
- project metadata
- lifecycle configurations

Basic pom.xml

```
None
<project>

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>

  <artifactId>employee-project</artifactId>

  <version>1.0-SNAPSHOT</version>

</project>
```

Maven Coordinates

Every Maven project contains:

Element	Meaning
groupId	Organization name
artifactId	Project name
version	Project version

Example

```
None
com.example:employee-project:1.0-SNAPSHOT
```

Maven Dependencies - Dependencies are external libraries required by the project.

Examples:

- Spring Boot
- Hibernate
- JUnit
- Log4j

Maven automatically downloads dependencies from repositories.

Dependency Workflow

```
None
pom.xml
 ↓
Maven Checks Local Repository
 ↓
If Dependency Missing
 ↓
Checks Remote/Central Repository
 ↓
Downloads Dependency
 ↓
Caches in Local Repository
```

Dependency Example

```
None
<dependencies>

  <dependency>

     <groupId>junit</groupId>

     <artifactId>junit</artifactId>

     <version>4.13.2</version>

  </dependency>

</dependencies>
```

Maven Repositories

Types of Repositories

Local Repository

Stored in:

```
None
.m2/repository
```

Contains cached dependencies.

Central Repository

Public repository managed by Maven community.

Contains open-source libraries.

Remote Repository

Organization-hosted repository.

Used in enterprise environments.

Maven Lifecycle - Lifecycle = predefined sequence of phases executed during build process.

Main Lifecycles

Lifecycle	Purpose
clean	Removes previous builds
default	Compiles/packages application
site	Generates documentation

Lifecycle Flow

```
clean
  ↓
validate
  ↓
compile
  ↓
test
  ↓
package
  ↓
install
  ↓
deploy
```

Build Phases

Phase	Purpose
validate	Validates project
compile	Compiles source code
test	Executes unit tests
package	Creates JAR/WAR
install	Installs artifact locally
deploy	Deploys artifact remotely

Maven Commands

Command	Purpose
mvn clean	Deletes target directory
mvn compile	Compiles source code
mvn test	Executes tests
mvn package	Generates artifact

mvn install	Installs artifact locally
mvn deploy	Deploys artifact

Maven Plugins - Plugins automate build-related tasks.

Without plugins:

- developers manually configure tasks

With plugins:

- tasks are automatically executed

Plugin Architecture

```
None
Lifecycle
 ↓
Phase
 ↓
Goal
 ↓
Plugin Execution
```

Common Plugins

Plugin	Purpose
compiler-plugin	Compiles code
surefire-plugin	Executes tests
jar-plugin	Creates JAR
war-plugin	Creates WAR

Plugin Example

```
None
<plugin>

  <groupId>org.apache.maven.plugins</groupId>
```

```
    <artifactId>maven-compiler-plugin</artifactId>

    <version>3.8.1</version>

</plugin>
```

Maven Goals - Goal = specific task execution inside Maven phase.

Examples:

- clean
- compile
- package
- install

Maven Profiles - Profiles allow environment-specific configurations.

Used for:

- development
- testing
- production

Different profiles can contain different:

- dependencies
- plugins
- configurations

Profile Example

```
None
<profiles>

    <profile>

        <id>dev</id>

    </profile>

</profiles>
```

Maven Artifact - Artifact = final generated output after build.

Types:

- JAR
- WAR
- EAR

Artifact Example

```
None
employee-project-1.0-SNAPSHOT.jar
```

Maven Build Internals

```
None
pom.xml
 ↓
Maven Parses XML
 ↓
Dependencies Resolved
 ↓
Plugins Loaded
 ↓
Lifecycle Executed
 ↓
Source Compiled
 ↓
Tests Executed
 ↓
Artifact Generated
```

Maven Archetype - Archetype is a Maven plugin used to generate project templates automatically.

It creates:

- standard folder structure
- pom.xml
- source folders
- test folders

Archetype Command

```
None
mvn archetype:generate
```

Maven Repository Flow

```
None
Developer Adds Dependency
      ↓
Maven Checks .m2 Repository
      ↓
If Missing
      ↓
Downloads from Central Repository
      ↓
Stores in Local Repository
```

Maven Dependency Scopes

Scope	Purpose
compile	Default dependency
provided	Available during runtime
runtime	Required only at runtime
test	Used for testing
system	System-level dependency

Snapshot Version - Snapshot represents:

```
None
latest development version
```

Example:

```
None
1.0-SNAPSHOT
```

Snapshot versions are updated frequently during development.

Maven Convention Over Configuration

Maven follows:

```
None
Convention over Configuration
```

Meaning:

- developers follow standard folder structure
- minimal manual configuration required

Standard Directory Structure

```
None
src/main/java
src/test/java
target/
```

Maven Inheritance

Maven supports inheritance through parent POM.

Allows:

- reusable configurations
- centralized dependency management
- plugin reuse

Inheritance Hierarchy

```
None
Settings
 ↓
Parent POM
 ↓
Project POM
```

Maven Build Profiles

Profiles customize builds for:

- different environments
- different databases

- testing vs production

Build Profile Flow

```
dev profile
  ↓
test profile
  ↓
prod profile
```

Maven Clean Lifecycle - Used to remove old generated files.

Clean Lifecycle Phases

Phase	Purpose
pre-clean	Executes before clean
clean	Removes target folder
post-clean	Executes after clean

Maven Site Lifecycle - Used for generating documentation.

Site Lifecycle Phases

Phase	Purpose
pre-site	Before documentation
site	Generate docs
post-site	After generation
site-deploy	Deploy documentation

Maven vs Ant

Maven	Ant
Declarative	Procedural

Uses conventions	Manual configuration
Lifecycle support	No lifecycle
Plugin support	Limited plugins
Reusable	Less reusable

Maven Integration with Jenkins

```
None
GitHub
 ↓
Jenkins
 ↓
mvn clean install
 ↓
Artifact Generated
 ↓
Deployment
```

Advantages of Maven

- automatic dependency management
- standardized structure
- reusable builds
- plugin ecosystem
- CI/CD integration
- easier maintenance
- supports enterprise projects

Limitations of Maven

- XML complexity
- dependency conflicts
- slower builds for huge projects
- learning curve for beginners

Real-Time Use Cases

Maven is used in:

- Spring Boot applications
- enterprise Java systems
- microservices
- DevOps CI/CD pipelines
- banking applications
- cloud deployments

End-to-End Maven Flow

```
Developer Writes Code
        ↓
pom.xml Defines Build
        ↓
Maven Reads Configuration
        ↓
Dependencies Downloaded
        ↓
Plugins Executed
        ↓
Build Lifecycle Starts
        ↓
Compilation
        ↓
Testing
        ↓
Packaging
        ↓
Artifact Generation
        ↓
Deployment
```